

Contents

Subject 1: Fundamentals of Programming Languages	4
Chapter 1: Rules of a Programming Language	4
1.1 Lexical, Syntactic and Semantic Rules	4
1.2 Pragmatics	5
1.3 Data Types and Type Systems	6
1.4 Expressions and Operators	6
Chapter 2: Expressions	7
2.1 Expression Evaluation	7
Chapter 3: Control Structures	8
3.1 Selection Statements (If-Else, Switch)	8
3.2 Iteration Statements (Loops)	9
3.3 Jump Statements	9
3.4 Exception Handling Concepts	10
Chapter 4: Types	10
4.1 Primitive Types	10
4.2 Composite Types	11
4.3 Type Conversion	12
4.4 Type Checking	12
Chapter 5: Memory Model	13
5.1 Stack vs Heap	13
5.2 Scope and Lifetime	13
5.3 Memory Management	14
Chapter 6: Exceptions	15
6.1 Exception Handling Mechanisms	15
6.2 Exception Safety	16
Practice Questions and Solutions	16
Cheat-Sheet: Key Definitions	17
Review Checklist	18
Chapter 1: Classes and Objects	18
1.1 Classes and Objects	18
1.2 Constructors	19
1.3 Static vs Instance Members	19
1.4 Object Relationships	20
Chapter 2: Encapsulation and Members	21
2.1 Encapsulation	21
2.2 Fields and Properties	22
2.3 Methods	23

Chapter 3: Polymorphism and Types	23
3.1 Polymorphism	23
3.2 Inheritance	24
3.3 Method Overriding	25
3.4 Abstract Classes and Interfaces	26
3.5 Type Casting and Conversion	27
Chapter 4: Memory Management	27
4.1 Stack and Heap in OOP	27
4.2 Object Lifecycle	28
4.3 Garbage Collection	29
4.4 Memory Issues	29
4.5 Smart Pointers (C++)	30
Chapter 5: Program Structure	30
5.1 Packages and Namespaces	30
5.2 Access Control Summary	31
5.3 Object-Oriented Design Principles	31
5.4 Design Patterns (Concepts)	32
Practice Questions and Solutions	33
MCQ 1: Inheritance	33
MCQ 2: Access Modifier	33
MCQ 3: Static	33
MCQ 4: Polymorphism	33
MCQ 5: GC	33
Short Answer 1	33
Short Answer 2	34
Short Answer 3	34
Short Answer 4	34
Cheat-Sheet: Key Definitions	34
Review Checklist	35
Chapter 1: Linear Data Structures	35
1.1 Arrays	35
1.2 Stacks	36
1.3 Queues	37
1.4 Linked Lists	38
1.5 Tower of Hanoi	39
Chapter 2: Trees and Hierarchical Structures	40
2.1 Tree Fundamentals	40
2.2 Binary Trees	40
2.3 Tree Traversals	41
2.4 Binary Search Trees (BST)	41
2.5 AVL Trees	42
2.6 Heaps	43

2.7 B+ Trees	44
Chapter 3: Hashing Techniques	45
3.1 Hash Tables	45
3.2 Hash Functions	46
3.3 Collisions	46
3.4 Chaining (Separate Chaining)	47
3.5 Open Addressing	47
3.6 HashMap in Java	48
Chapter 4: Graphs	48
4.1 Graph Fundamentals	48
4.2 Graph Representations	49
4.3 Breadth-First Search (BFS)	50
4.4 Depth-First Search (DFS)	51
4.5 Minimum Spanning Tree (MST)	51
4.6 Shortest Path Algorithms	52
4.7 Topological Sort	53
4.8 Cycle Detection	54
Practice Questions and Solutions	54
MCQ 1: Array Access	54
MCQ 2: Stack	54
MCQ 3: BST	55
MCQ 4: AVL	55
MCQ 5: Heap	55
MCQ 6: Hash Collision	55
MCQ 7: BFS	55
MCQ 8: Dijkstra	55
MCQ 9: MST	55
MCQ 10: Topological Sort	55
Short Answer 1	56
Short Answer 2	56
Short Answer 3	56
Short Answer 4	56
Short Answer 5	56
Short Answer 6	56
Cheat-Sheet: Time Complexities	57
Data Structures	57
Graph Algorithms	57
Review Checklist	57
Arrays and Lists	57
Stacks and Queues	57
Trees	58
Hashing	58
Graphs	58

Subject 1: Fundamentals of Programming Languages

Chapter 1: Rules of a Programming Language

1.1 Lexical, Syntactic and Semantic Rules

Definition / Theoretical Background

Programming languages are defined by three layers of rules that govern how code is written, interpreted, and executed.

Lexical Rules (The “Spelling” of Language) Lexical rules define how characters are grouped into meaningful tokens—the smallest units of a program.

Token Types:

Token Type	Description	Examples
Keywords/Reserved Words	Words with special meaning, cannot be reused	<code>if, while, return, class</code>
Identifiers	User-defined names for variables, functions	<code>count, calculateTotal, maxValue</code>
Literals	Fixed values that don't change	<code>42, 3.14, "Hello", true</code>
Operators	Operations to perform	<code>+, -, *, /, ==, &&</code>
Delimiters/Punctuation	Structural separators	<code>(,), {, }, ;, ,</code>
Comments	Non-executable documentation	<code>// line comment, /* block comment */</code>

Syntactic Rules (The “Grammar” of Language) Syntactic rules define how tokens combine to form valid statements and programs.

Key Concepts: - **Context-Free Grammar (CFG):** Most programming languages use CFG for syntax - **Derivation:** Starting from the start symbol, repeatedly apply production rules until you get the token sequence - **Parse Tree:** Visual representation of how a program can be derived from the grammar

Semantic Rules (The “Meaning” of Language) Semantic rules define what programs mean—what they actually do when executed.

Aspect	Example
Type Rules	<code>int x = "hello";</code> – type mismatch error
Scope Rules	Using a variable before declaration
Context Sensitivity	<code>*</code> means multiplication for numbers, dereference for pointers
Evaluation Order	<code>a + b * c</code> – which addition happens first?

Traps

Trap 1: Reserved Words vs Identifiers - In C/C++, `sizeof` is a keyword and cannot be used as variable name - In some languages like SQL, you can sometimes escape reserved words

Trap 2: Unicode and Identifiers - Python allows Unicode identifiers (PEP 3131): `café = 42` is valid - Not all languages support Unicode identifiers

Trap 3: Context-Sensitive Syntax - In JavaScript, `/` can mean division OR regex depending on context: `y / 2` = division - `/[a-z]+/` = regex literal

Trap 4: Invisible Characters in Source - Non-breaking space (U+200B) can appear invisible in code but cause compilation errors

Practice Questions

MCQ 1: What type of rule determines whether `int x = "hello";` is valid? - A) Lexical rule - B) Syntactic rule - C) Semantic rule - D) Pragmatic rule

MCQ 2: Which of the following is NOT a valid token type? - A) Identifier - B) Reserved word - C) Semantic marker - D) Operator

Short Answer: Explain why `x = y` is syntactically valid but semantically invalid when `x` is `int` and `y` is `float`.

Solutions

MCQ 1: C) Semantic rule – type checking happens during semantic analysis.

MCQ 2: C) “Semantic marker” is not a standard token type.

Short Answer: The assignment `x = y` follows correct syntactic structure. However, assigning a `float` to an `int` variable violates semantic rules (type compatibility).

1.2 Pragmatics

Definition

Pragmatics refers to the practical aspects of writing code—conventions, style, and formatting that make code readable and maintainable.

Naming Conventions

Style	Example	Used In
camelCase	<code>maxValue</code> , <code>calculateTotal</code>	Java, JavaScript
PascalCase	<code>MaxValue</code> , <code>CalculateTotal</code>	C#, TypeScript classes
snake_case	<code>max_value</code> , <code>calculate_total</code>	Python, C
SCREAMING_SNAKE_CASE	<code>MAX_VALUE</code> , <code>BUFFER_SIZE</code>	Constants (all languages)

1.3 Data Types and Type Systems

Definition

A **data type** defines the set of values and operations that can be performed on those values.

Type System Classifications

Classification	Description	Examples
Static Typing	Types checked at compile time	Java, C, C++
Dynamic Typing	Types checked at runtime	Python, JavaScript, Ruby
Strong Typing	Automatic conversions are rare/minimal	Java, Python
Weak Typing	Automatic conversions are common	C, JavaScript, PHP

Primitive Data Types

Type	Description	Typical Size
Integer	Whole numbers	4 bytes (32-bit)
Float/Real	Decimal numbers	4 bytes (32-bit)
Double	Higher precision decimal	8 bytes (64-bit)
Character	Single character	1 byte
Boolean	True/False values	1 byte

Traps

Trap 1: Floating Point Precision - $0.1 + 0.2 \neq 0.3$ in binary floating point - Never compare floats with ==

Trap 2: Integer Overflow - `int max = 2147483647; max + 1 = -2147483648` (wraps around)
- Unsigned integers wrap around, signed integers are undefined

Trap 3: Implicit Type Conversion - $7 / 2 = 3$ in integer division (C/Java) - $7.0 / 2 = 3.5$ in floating point division

1.4 Expressions and Operators

Definition

An **expression** is a combination of operands and operators that produces a value.

Operator Precedence (High to Low)

Priority	Operators	Associativity
1	() (parentheses)	N/A
2	!, - (unary)	Right to Left
3	*, /, %	Left to Right

Priority	Operators	Associativity
4	+, -	Left to Right
5	<, <=, >, >=	Left to Right
6	==, !=	Left to Right
7	&&	Left to Right
8	\ \	Left to Right
9	= (assignment)	Right to Left

Key Concepts

Short-Circuit Evaluation: - **&&**: If left side is false, right side is not evaluated - **\|**: If left side is true, right side is not evaluated

Ternary Operator: - `condition ? value_if_true : value_if_false`

Chapter 2: Expressions

2.1 Expression Evaluation

Definition

An **expression** evaluates to a value. The process of determining this value is called expression evaluation.

Expression Types

Type	Example	Result
Arithmetic	<code>3 + 4 * 2</code>	<code>11</code>
Relational	<code>5 > 3</code>	<code>true</code>
Logical	<code>true && false</code>	<code>false</code>
Assignment	<code>x = 5</code>	5 (value), x becomes 5
Compound	<code>a += b</code>	Adds b to a

Evaluation Order Rules

1. **Precedence:** Higher precedence operators evaluate first
2. **Associativity:** Equal precedence evaluates left-to-right (except assignment)
3. **Short-circuit:** Logical operators may skip right operand

Traps

Trap 1: Assignment as Expression - In C/Java: `a = b = c = 0` assigns right to left - Value of expression is the assigned value

Trap 2: Side Effects and Evaluation Order

```
int i = 0;
int arr[2] = {0, 0};
arr[i++] = i++; // Undefined behavior in C/C++!
```

Chapter 3: Control Structures

3.1 Selection Statements (If-Else, Switch)

If-Else Structure

```
if (condition1)
    statement1
else if (condition2)
    statement2
else
    statement3
```

Switch Statement

```
switch (expression) {
    case constant1:
        statements;
        break;
    case constant2:
        statements;
        break;
    default:
        statements;
}
```

Key Rules

- Switch expression must evaluate to integral or enumerated type (Java) or any discrete type (C)
- Case constants must be compile-time constants
- Break prevents fall-through

Traps

Trap 1: Fall-Through Without Break

```
switch (x) {
    case 1: printf("One");
    case 2: printf("Two"); // No break - falls through!
    case 3: printf("Three");
}
```


If $x = 1$, output is “OneTwoThree”

Trap 2: Switch Without Default - Missing default means no action for unmatched values

3.2 Iteration Statements (Loops)

While Loop

```
while (condition) {  
    statements;  
}
```

Entry-condition: Loop body executes only if condition is true.

Do-While Loop

```
do {  
    statements;  
} while (condition);
```

Exit-condition: Loop body executes at least once.

For Loop

```
for (init; condition; update) {  
    statements;  
}
```

Infinite Loop

```
for (;;) { }      // No condition = infinite  
while (1) { }     // Condition always true
```

Loop Control

Statement	Effect
<code>break</code>	Exit innermost loop/switch
<code>continue</code>	Skip to next iteration

3.3 Jump Statements

Return

- Exits function, optionally returning a value

Goto

- Transfers control to labeled statement
- Considered harmful in structured programming

Traps

Goto and Labels:

```
if (error)
    goto cleanup;

cleanup:
    // Code to execute on error or normal exit
```

3.4 Exception Handling Concepts

Exception Categories

Category	Description	Examples
Syntactic	Language syntax violations	Missing semicolon
Semantic	Logic errors, type errors	Uninitialized variable
Runtime	Errors during execution	Division by zero, null pointer

Defensive Programming

```
// Check all function return values
int result = divide(a, b);
if (result == ERROR_VALUE) {
    handle_error();
}
```

Chapter 4: Types

4.1 Primitive Types

Integer Types

Type	Typical Range
char	-128 to 127 or 0 to 255
short	-32,768 to 32,767
int	-2,147,483,648 to 2,147,483,647
long	-9.2e18 to 9.2e18
long long	Much larger range

Floating Point Types

Type	Precision	Typical Range
float	6-7 significant digits	1.2e-38 to 3.4e38
double	15-16 significant digits	2.2e-308 to 1.8e308

4.2 Composite Types

Arrays

Definition: A collection of elements of the same type, accessed by index.

- Index starts at 0 (in most languages)
- Size is fixed at declaration (static) or can grow (dynamic)
- Contiguous memory allocation

Structures (struct)

Definition: A composite type that groups heterogeneous data members.

```
struct Point {  
    int x;  
    int y;  
};
```

Pointers

Definition: A variable that stores the memory address of another variable.

Memory:

Address:	0x1000	0x1004	0x1008
Value:	42	99	...

^
|
Pointer p
stores 0x1004

Traps

Trap 1: Pointer Arithmetic

```
int arr[5] = {1, 2, 3, 4, 5};  
int *p = arr;           // p points to arr[0]  
p = p + 2;               // p now points to arr[2]
```

Trap 2: Dangling Pointers

```
int *p = &x;  
free(p);                 // Memory freed  
// p now points to freed memory - DANGLING!
```

4.3 Type Conversion

Implicit Conversion (Coercion)

Automatic conversion by the compiler from one type to another.

Rules: - Smaller to larger types (promotion) - Integer to floating point - Derived to base class (inheritance)

Explicit Conversion (Casting)

Manual conversion specified by programmer.

```
double x = 3.14;  
int y = (int) x;    // y = 3 (truncates)
```

Type Casting Dangers

Cast Type	Example	Risk
Narrowing	double to int	Data loss
Widening	int to double	Safe
Pointer cast	(int*)voidp	Undefined behavior

4.4 Type Checking

Static Type Checking

- Performed at compile time
- Catches type errors before execution
- Example: Java, C++

Dynamic Type Checking

- Performed at runtime
- More flexible but errors caught later
- Example: Python, JavaScript

Type Compatibility

System	Compatible Types
Nominal	Must be explicitly declared related
Structural	Compatible if same structure

Chapter 5: Memory Model

5.1 Stack vs Heap

Stack Memory

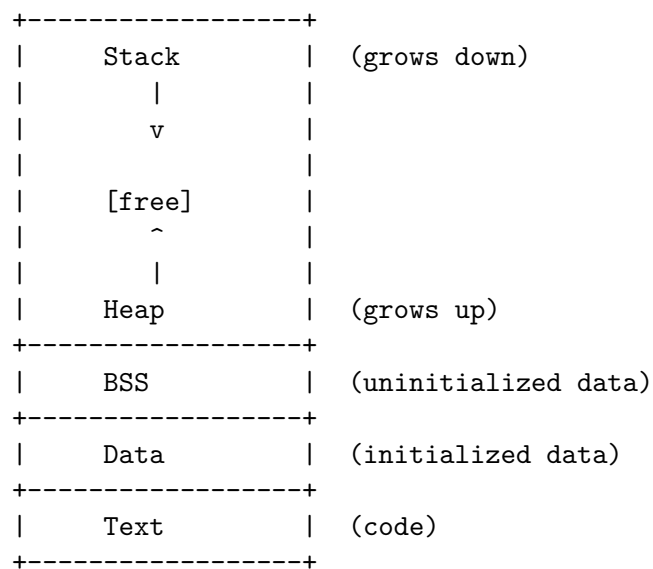
Property	Description
Allocation	Automatic (function call)
Deallocation	Automatic (function return)
Size	Fixed, limited
Access	Fast
Lifetime	Scoped to function

Heap Memory

Property	Description
Allocation	Manual (malloc/new)
Deallocation	Manual (free/delete) or Garbage Collector
Size	Large, flexible
Access	Slower than stack
Lifetime	Until explicitly freed

Memory Layout

High Addresses



Low Addresses

5.2 Scope and Lifetime

Scope Types

Type	Description
Block scope	Within {} braces
Function scope	Entire function
File scope	Entire file (global)
Class scope	Within class

Variable Lifetime

Storage	Lifetime
Automatic	Until block exits
Static	Entire program
Dynamic	Until freed

Static Variables

```
void counter() {
    static int count = 0; // Initialized once, retains value
    count++;
}
```

5.3 Memory Management

Manual Management (C/C++)

Allocation:

```
int *p = (int*)malloc(sizeof(int));
```

Deallocation:

```
free(p);
p = NULL; // Prevent dangling pointer
```

Automatic Management (Java/Python)

- Garbage collector automatically frees unused objects
- Mark-and-sweep, reference counting, generational GC

Memory Errors

Error	Description	Consequence
Leak	Not freeing memory	Eventually out of memory
Double free	Freeing twice	Undefined behavior
Use after free	Using freed memory	Undefined behavior

Error	Description	Consequence
Buffer overflow	Writing past array	Corrupts memory

Chapter 6: Exceptions

6.1 Exception Handling Mechanisms

Exception Definition

An **exception** is an event that occurs during program execution that disrupts normal flow.

Exception Propagation

```
main() {
    f1() {
        f2() {
            throw error;
        }
    }
}
// Exception propagates: f2 -> f1 -> main
```

Catch Order

- Exceptions are caught by the first matching catch block
- More specific exceptions must come before general ones

Traps

Trap 1: Catching Wrong Exception Type

```
try {
    // code
} catch (Exception e) {
    // Catches ALL exceptions
    // May hide specific errors
}
```

Trap 2: Swallowing Exceptions

```
try {
    // code
} catch (Exception e) {
    // Do nothing - exception silently ignored!
}
```

6.2 Exception Safety

Exception Safety Levels

Level	Guarantee
No-throw	Operations never throw
Strong	Operation succeeds or has no side effects
Basic	Objects remain in valid state
No guarantee	Nothing guaranteed

RAII (Resource Acquisition Is Initialization)

```
class FileGuard {  
    FILE* file;  
public:  
    FileGuard(const char* name) {  
        file = fopen(name, "w");  
    }  
    ~FileGuard() {  
        if (file) fclose(file); // Auto-closes  
    }  
};
```

Practice Questions and Solutions

MCQ 1: Lexical Rules

Which is NOT a lexical rule consideration? - A) Token classification - B) Identifier naming - C) Grammar production rules - D) Whitespace handling

Answer: C) Grammar production rules (that's syntactic)

MCQ 2: Type System

Which language uses dynamic typing? - A) Java - B) C - C) Python - D) C++

Answer: C) Python

MCQ 3: Memory

What happens when you free the same memory twice? - A) Memory is set to zero - B) Undefined behavior - C) Nothing happens - D) Error is thrown

Answer: B) Undefined behavior

Short Answer 1

Explain the difference between static and dynamic type checking.

Answer: Static type checking happens at compile time - the compiler verifies types before execution. Dynamic type checking happens at runtime - types are checked as code executes. Static catches errors earlier but is less flexible; dynamic is more flexible but errors may occur at runtime.

Short Answer 2

Why is $0.1 + 0.2 \neq 0.3$ in floating point?

Answer: Because floating point numbers are represented in binary (base 2), not decimal (base 10). The value 0.1 in binary is a repeating fraction: 0.000110011001100... Similarly 0.2 is 0.001100110011... These cannot be represented exactly, so rounding errors occur in the calculation, making $0.1 + 0.2$ slightly different from 0.3.

Short Answer 3

Explain stack vs heap memory.

Answer: Stack is automatic memory managed by the compiler, used for local variables and function calls. It's fast but limited in size and follows LIFO discipline. Heap is manual/dynamic memory allocated explicitly (malloc/new) or by garbage collectors. It's larger but slower, and the programmer must manage its lifetime to avoid leaks.

Cheat-Sheet: Key Definitions

Term	Definition
Lexical Rules	Rules for forming tokens (spelling)
Syntactic Rules	Rules for combining tokens (grammar)
Semantic Rules	Rules for meaning (type, scope)
Token	Smallest meaningful unit of a program
Parse Tree	Visual derivation showing syntax structure
Static Typing	Type checking at compile time
Dynamic Typing	Type checking at runtime
Strong Typing	Automatic conversions are rare
Weak Typing	Automatic conversions are common
Pragmatics	Code style and convention
Scope	Region where variable is accessible
Lifetime	Time during which variable exists
Stack Memory	Automatic, fast, limited
Heap Memory	Manual/GC, large, slower
Pointer	Variable storing memory address
Exception	Runtime error disrupting normal flow
RAII	Resource management via constructor/destructor

Review Checklist

- ☐ Can you distinguish lexical, syntactic, and semantic rules?
 - ☐ Do you understand type system classifications?
 - ☐ Can you trace expression evaluation with operator precedence?
 - ☐ Do you know the differences between loop types?
 - ☐ Can you explain stack vs heap memory?
 - ☐ Do you understand exception propagation?
 - ☐ Can you identify memory errors (leak, dangling, overflow)? # Subject 2: Object-Oriented Programming Languages
-

Chapter 1: Classes and Objects

1.1 Classes and Objects

Definition / Theoretical Background

Class: A blueprint or template that defines the structure and behavior of objects. It specifies: - **Fields** (attributes/properties): Data that objects store - **Methods** (behaviors): Operations that objects can perform

Object: An instance of a class - a concrete entity created from the blueprint.

Concept	What It Is	Example
Class	Blueprint	<code>class Dog { ... }</code>
Object	Instance of class	<code>Dog buddy = new Dog();</code>
Instantiation	Creating object from class	<code>new Dog()</code>
State	Values in object's fields	<code>buddy.name = "Buddy"</code>
Identity	Unique identifier	Each object has unique address
Behavior	Methods object can perform	<code>buddy.bark()</code>

The **new** Keyword

When you create an object with **new**: 1. Allocates memory for the object (usually on the heap) 2. Initializes fields to default values (0, null, false) 3. Calls the constructor method

this Reference

this is a reference to the current object. Used to: - Distinguish instance variables from parameters with same name - Pass current object as argument to other methods - Return current object from a method

1.2 Constructors

Definition

A **constructor** is a special method that initializes new objects. It has the same name as the class and no return type.

Constructor Types

Type	Description	When Called
Default Constructor	No parameters, initializes defaults	When no constructor defined
Parameterized Constructor	Accepts parameters	When explicit parameters provided
Copy Constructor	Creates copy of another object	When object passed by value (C++)

Constructor Overloading

Multiple constructors with different parameter lists:

```
class Point {  
    int x, y;  
  
    Point() { x = 0; y = 0; }           // No-arg constructor  
    Point(int x, int y) { this.x = x; this.y = y; } // Parameterized  
    Point(Point p) { x = p.x; y = p.y; } // Copy constructor  
}
```

Traps

Trap 1: Forgetting to Initialize - Instance variables have default values (0, null, false) - But relying on defaults can cause bugs

Trap 2: Constructor Calling Constructor - In Java, use `this(...)` to call another constructor - Must be first statement

1.3 Static vs Instance Members

Instance Members

Property	Belong To	Access Via	Exist When
Instance variable	Object	Object reference	Object exists
Instance method	Object	Object reference	Object exists

Static Members (Class Members)

Property	Belong To	Access Via	Exist When
Static variable	Class	Class name or object	Program start to end
Static method	Class	Class name or object	Program start to end

Key Differences

Aspect	Static	Instance
Copies	One copy per class	One copy per object
Access	Via class name or object	Via object reference
Lifetime	Entire program	Object lifetime
This reference	None	Has <code>this</code>

When to Use Static

- Constants (mathematical values, configuration)
- Utility functions (no object state needed)
- Factory methods
- Main method (program entry point)

1.4 Object Relationships

Association

General relationship where objects know about each other.

Student ---> Course
(A student enrolls in courses)

Aggregation

“Has-a” relationship with shared ownership. Part can exist without whole.

University "has" Department
(Department can exist without university)

Composition

“Has-a” relationship with exclusive ownership. Part cannot exist without whole.

Car "has" Engine
(Engine cannot exist without car)

Difference Table

Relationship	Ownership	Lifetime	Example
Association	None	Independent	Student takes Course
Aggregation	Shared	Independent	University has Dept
Composition	Exclusive	Same	Car has Engine

Chapter 2: Encapsulation and Members

2.1 Encapsulation

Definition

Encapsulation is the bundling of data and methods that operate on that data within a single unit (class). It also restricts direct access to some components.

Purpose / Benefits

1. **Data Hiding:** Internal details hidden from outside
2. **Modularity:** Self-contained units
3. **Maintainability:** Changes internal implementation don't affect users
4. **Reusability:** Encapsulated components can be reused

Access Modifiers

Modifier	Same Class	Same Package	Subclass	Other Packages
private	Yes	No	No	No
default (package-private)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Information Hiding Levels

public: Fully accessible
protected: Accessible to subclasses and package
private: Only within the class

Getters and Setters

Methods that provide controlled access to private fields:

```
private int age;

public int getAge() { return age; }
public void setAge(int age) {
    if (age >= 0) this.age = age;
}
```

Traps

Trap 1: Breaking Encapsulation

```
// Returns mutable object - external code can modify internal state!
public List<String> getNames() {
    return names; // BAD! Returns reference to internal list
}

// FIX: Return a copy
public List<String> getNames() {
    return new ArrayList<>(names);
}
```

Trap 2: Over-Exposure - Making everything public defeats the purpose - Only expose what's necessary

2.2 Fields and Properties

Fields (Instance Variables)

- Declared at class level
- Each object has its own copy
- Store object state

Properties (C#, Python)

A property is a combination of a field and methods:

```
private int _age; // Backing field

public int Age { // Property
    get { return _age; }
    set { _age = value; }
}
```

Final Fields (Constants)

```
final double PI = 3.14159;
final int MAX_SIZE = 100;
```

Once assigned, cannot be changed.

Transient Fields

Marked with `transient` keyword - not serialized (Java).

2.3 Methods

Method Signature

The combination of method name and parameter types.

```
int add(int a, int b) // Signature: add(int, int)
```

Return Values

Return Type	Description
Primitive	Return value directly
Object	Return reference
void	No return value
var (Java 10+)	Type inferred

Method Overloading

Multiple methods with same name but different parameters:

```
int add(int a, int b) // add(int, int)
double add(double a, double b) // add(double, double)
int add(int a, int b, int c) // add(int, int, int)
```

Not overloading:

```
int add(int a, int b) // Return type doesn't matter!
void add(int a, int b) // ERROR: same signature
```

Parameter Passing

Type	What Gets Passed	Effect
Primitive	Copy of value	Changes don't affect caller
Object	Copy of reference	Changes to fields affect caller
Reference parameter	Reference itself	Can change caller's variable

Chapter 3: Polymorphism and Types

3.1 Polymorphism

Definition

Polymorphism means “many forms” - the ability of one interface to represent different underlying forms (types).

Types of Polymorphism

Type	Description	When
Ad-hoc (Overloading)	Same name, different signatures	Compile time
Parametric	Generic types/Generics	Compile time
Subtype (Inheritance)	Same interface, different implementations	Runtime

Compile-Time vs Runtime Polymorphism

Aspect	Compile-Time	Runtime
Decision	Method to call	Which implementation
Mechanism	Overloading, templates	Virtual methods
Binding	Early binding	Late binding (dynamic dispatch)

3.2 Inheritance

Definition

Inheritance allows a class (subclass) to inherit fields and methods from another class (superclass).

Inheritance Relationship

Superclass (Parent, Base)
|
v
Subclass (Child, Derived)

Inheritance Types

Type	Description	Example
Single	One parent, one child	Class -> Subclass
Multiple	Multiple parents	Not in Java
Multilevel	Chain of inheritance	A -> B -> C
Hierarchical	One parent, many children	Animal -> Dog, Cat
Hybrid	Combination	A -> B, C; D -> B, C

Inheritance vs Composition

Inheritance (IS-A)	Composition (HAS-A)
Tightly coupled	Loosely coupled
Fixed at compile time	Can change at runtime
Single inheritance preferred	Favored in design

Prefer composition over inheritance because: - More flexible - Easier to test - Less fragile

protected Access

Protected members are accessible: - Within the class - Within same package - In subclasses

3.3 Method Overriding

Definition

Method overriding allows a subclass to provide a specific implementation of a method already defined in its superclass.

Override Requirements

Requirement	Description
Same name	Method name must match
Same parameters	Parameter list must match
Same return type	Or covariant return
Cannot override static	Static methods are not virtual
Access level	Cannot be more restrictive

super Keyword

Used to call parent class methods/constructors:

```
class Parent {
    void display() { System.out.println("Parent"); }
}

class Child extends Parent {
    @Override
    void display() {
        super.display(); // Call parent's display
        System.out.println("Child");
    }
}
```

Traps

Trap: Overloading vs Overriding

```
class Parent {
    void display(int x) { } // Overload, not override!
}

class Child extends Parent {
    void display() { } // This is a NEW method, not overriding
    // display() and display(int) are different methods
}
```

3.4 Abstract Classes and Interfaces

Abstract Class

A class that cannot be instantiated - serves as a base for subclasses.

```
abstract class Shape {  
    abstract double area(); // Abstract method - no implementation  
}  
  
class Circle extends Shape {  
    double area() { return Math.PI * r * r; }  
}
```

Abstract Class	Concrete Class
Can have abstract methods	No abstract methods
Cannot be instantiated	Can be instantiated
Can have implemented methods	All methods must be implemented

Interface

A contract that defines a set of methods that implementing classes must provide.

```
interface Drawable {  
    void draw();  
}  
  
class Circle implements Drawable {  
    public void draw() { /* implementation */ }  
}
```

Interface vs Abstract Class

Aspect	Interface	Abstract Class
Methods	Abstract (until Java 8)	Can have both
Fields	Only constants	Any type
Multiple implements	Yes	No (single inheritance)
Constructors	No	Yes
Access modifiers	Public only	Any

Default Methods (Java 8+)

```
interface Calculator {  
    int add(int a, int b);  
}
```

```
default int subtract(int a, int b) {  
    return a - b; // Default implementation  
}  
}
```

3.5 Type Casting and Conversion

Upcasting (Widening)

Automatic conversion from subclass to superclass.

```
Dog dog = new Dog();  
Animal animal = dog; // Upcasting - always safe
```

Downcasting (Narrowing)

Explicit conversion from superclass to subclass.

```
Animal animal = new Dog();  
Dog dog = (Dog) animal; // Downcasting - needs cast
```

instanceof Operator

```
if (animal instanceof Dog) {  
    Dog dog = (Dog) animal;  
    dog.bark();  
}
```

Traps

Trap: ClassCastException

```
Animal animal = new Cat();  
Dog dog = (Dog) animal; // ClassCastException at runtime!
```

Safe Pattern:

```
if (animal instanceof Dog) {  
    Dog dog = (Dog) animal; // Safe after instanceof check  
}
```

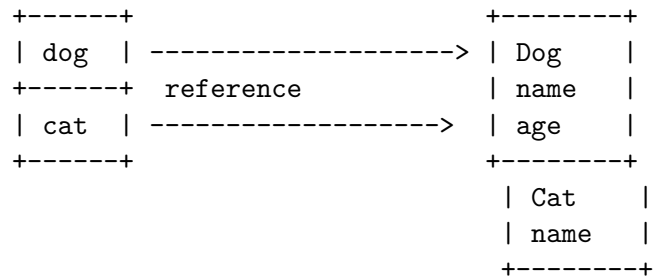
Chapter 4: Memory Management

4.1 Stack and Heap in OOP

Object Storage

STACK

HEAP



Stack (Automatic Memory)

Property	Description
What it stores	Local variables, method calls, references
Allocation	Automatic when entering method
Deallocation	Automatic when exiting method
Size	Limited
Speed	Fast

Heap (Dynamic Memory)

Property	Description
What it stores	Objects, dynamically allocated data
Allocation	Manual (new) or GC
Deallocation	Manual (delete/free) or GC
Size	Large
Speed	Slower

4.2 Object Lifecycle

Creation

```
Dog buddy = new Dog("Buddy", 3);
```

1. `new` allocates heap memory
2. Constructor initializes fields
3. Reference stored in variable

Reaching Unreachable Objects

An object becomes eligible for GC when: - No live references point to it - All references are null - Circular references with no external reference

GC Process

1. **Mark:** Identify reachable objects

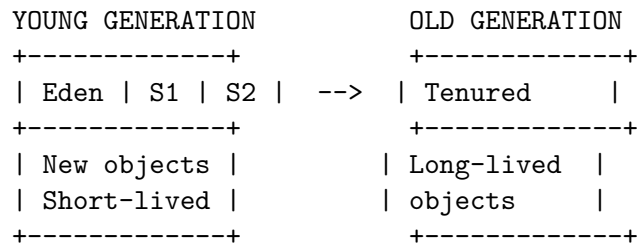
2. **Sweep:** Collect unreachable objects
 3. **Compact:** Move objects to reduce fragmentation
-

4.3 Garbage Collection

Types of GC

Type	Description	Example
Reference Counting	Count references to each object	Python (partially)
Mark-and-Sweep	Mark reachable, sweep rest	Java (early)
Generational	Young/old generations	Java (modern)
Incremental	Small pauses frequently	G1GC

Generational GC



GC Triggers

- When young generation fills
 - When heap needs space
 - When application idle
 - Manual `System.gc()` (not guaranteed)
-

4.4 Memory Issues

Memory Leak

Objects that are no longer needed but still referenced.

```
List<String> list = new ArrayList<>();
while (true) {
    list.add("Leak!"); // Never removed
}
```

Causes: - Unclosed resources (files, connections) - Cached objects without eviction - Static references to objects

Null Reference

```
Dog dog = new Dog("Buddy", 3);  
dog = null; // Object now unreachable  
// GC will eventually reclaim it
```

Dangling Reference

Reference to freed memory (C/C++ without GC).

4.5 Smart Pointers (C++)

RAII (Resource Acquisition Is Initialization)

```
class SmartPtr {  
    Dog* ptr;  
public:  
    SmartPtr(Dog* p) : ptr(p) {}  
    ~SmartPtr() { delete ptr; } // Auto-delete  
};
```

Types of Smart Pointers

Type	Description	When to use
unique_ptr	Exclusive ownership	Single owner
shared_ptr	Reference counting	Multiple owners
weak_ptr	Non-owning reference	Break cycles

Chapter 5: Program Structure

5.1 Packages and Namespaces

Package (Java)

A namespace for organizing classes.

```
package com.company.project;
```

Namespace (C++/C#)

```
namespace MathUtils {  
    int add(int a, int b) { return a + b; }  
}
```

```
// Usage
MathUtils::add(3, 4);
```

Import/Using

```
import java.util.List;    // Single class
import java.util.*;       // All classes in package

using System.Collections.Generic; // C#
```

5.2 Access Control Summary

Modifier	Class	Package	Subclass	World
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default	Yes	Yes	No	No
private	Yes	No	No	No

5.3 Object-Oriented Design Principles

SOLID Principles

Principle	Description
Single Responsibility	One class, one reason to change
Open/Closed	Open for extension, closed for modification
Liskov Substitution	Subtypes must be substitutable for base types
Interface Segregation	Many specific interfaces > one general
Dependency Inversion	Depend on abstractions, not concretions

Liskov Substitution Principle (LSP)

```
// Square IS-A Rectangle, but...
class Rectangle {
    void setWidth(int w) { width = w; }
    void setHeight(int h) { height = h; }
}

class Square extends Rectangle {
    void setWidth(int w) { width = height = w; }
    void setHeight(int h) { width = height = h; }
}
```

```
// Violation:
Rectangle r = new Square();
r.setWidth(5); // Sets height to 5 too!
r.setHeight(10); // Sets width to 10!
```

Dependency Inversion

High-level modules should not depend on low-level modules. Both should depend on abstractions.

5.4 Design Patterns (Concepts)

Creational Patterns

Pattern	Purpose
Singleton	One instance only
Factory Method	Create objects without specifying class
Abstract Factory	Create families of related objects
Builder	Construct complex objects step by step

Singleton Example Concept

```
class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Structural Patterns

Pattern	Purpose
Adapter	Convert interface to another
Decorator	Add behavior dynamically
Facade	Simplified interface

Behavioral Patterns

Pattern	Purpose
Observer	Notify dependents of state changes
Strategy	Interchangeable algorithms
Command	Encapsulate operation as object

Practice Questions and Solutions

MCQ 1: Inheritance

What type of inheritance is NOT directly supported by Java? - A) Single inheritance - B) Multilevel inheritance - C) Multiple inheritance - D) Hierarchical inheritance

Answer: C) Multiple inheritance (Java uses interfaces instead)

MCQ 2: Access Modifier

Which modifier makes a member accessible only within the same package? - A) private - B) protected - C) default (package-private) - D) public

Answer: C) default

MCQ 3: Static

What is wrong with accessing static method through instance? - A) Nothing, it works - B) It causes compilation error - C) It's confusing but works - D) Depends on language

Answer: A) It works but is confusing - Should use `ClassName.method()`

MCQ 4: Polymorphism

Which type of polymorphism is achieved through method overriding? - A) Ad-hoc polymorphism - B) Parametric polymorphism - C) Subtype polymorphism - D) None

Answer: C) Subtype polymorphism

MCQ 5: GC

An object is eligible for garbage collection when: - A) All references to it are null - B) No live threads can reach it - C) It's unreachable - D) All of the above

Answer: D) All of the above

Short Answer 1

Explain the difference between aggregation and composition.

Answer: Aggregation represents a “has-a” relationship where the part can exist independently of the whole (e.g., a Department can exist without a University). Composition represents a stronger “has-a” relationship where the part cannot exist without the whole (e.g., an Engine cannot exist without a Car). In composition, if the whole is destroyed, the parts are destroyed too.

Short Answer 2

Why should you prefer composition over inheritance?

Answer: Composition is preferred because: (1) It provides looser coupling - the contained object can be changed at runtime; (2) It's more flexible and easier to test - you can mock the contained object; (3) It avoids issues with inheritance like the fragile base class problem; (4) It follows the "favor has-a over is-a" design principle. Inheritance creates tight coupling between parent and child classes.

Short Answer 3

Explain the difference between static binding and dynamic binding.

Answer: Static binding (early binding) occurs at compile time and is resolved statically. Method overloading uses static binding. Dynamic binding (late binding) occurs at runtime and is resolved based on the actual object type. Method overriding uses dynamic binding - the JVM determines which implementation to call based on the actual object type, not the reference type.

Short Answer 4

What is the purpose of the `this` keyword in OOP?

Answer: The `this` keyword refers to the current object instance. It serves three purposes: (1) To disambiguate instance variables from parameters with the same name (e.g., `this.x = x`); (2) To pass the current object as an argument to another method; (3) To enable method chaining (returning `this` from a method). It provides a way for an object to reference itself.

Cheat-Sheet: Key Definitions

Term	Definition
Class	Blueprint/template defining structure and behavior
Object	Instance of a class
Encapsulation	Bundling data and methods, restricting access
Inheritance	Subclass acquiring parent's fields and methods
Polymorphism	Same interface, different implementations
Abstraction	Hiding complexity, showing only essentials
Constructor	Special method that initializes new objects
this	Reference to current object
super	Reference to parent class
static	Belongs to class, not instance
final	Cannot be changed/overridden
abstract	Cannot be instantiated, defines interface
interface	Contract specifying method signatures
Garbage Collection	Automatic memory management
Stack	Automatic memory for local variables
Heap	Dynamic memory for objects

Review Checklist

- ☐ Can you explain class vs object?
 - ☐ Do you understand encapsulation and access modifiers?
 - ☐ Can you distinguish inheritance types?
 - ☐ Do you know the difference between overriding and overloading?
 - ☐ Can you explain abstract class vs interface?
 - ☐ Do you understand static vs instance members?
 - ☐ Can you describe memory management in OOP?
 - ☐ Do you know SOLID principles?
 - ☐ Can you explain aggregation vs composition?
 - ☐ Do you understand GC and when objects become eligible? # Subject 3: Data Structures and Algorithms
-

Chapter 1: Linear Data Structures

1.1 Arrays

Definition

An **array** is a contiguous block of memory containing elements of the same type, accessed by integer indices.

Key Properties

- **Fixed size** (static) or **dynamic** (growable)
- **Contiguous memory** - all elements stored side-by-side
- **O(1) random access** - any element accessed in constant time
- **Homogeneous** - all elements are the same type

Memory Address Calculation

Element at index `i`: `address = base_address + i * sizeof(element)`

Example with int (4 bytes):

`base_address = 0x1000`

`arr[3] address = 0x1000 + 3 * 4 = 0x100C`

Array Operations Complexity

Operation	Time Complexity	Notes
Access by index	O(1)	Direct calculation
Search (unsorted)	O(n)	Linear scan
Search (sorted)	O(log n)	Binary search

Operation	Time Complexity	Notes
Insert at end	$O(1)$ amortized	Dynamic arrays only
Insert at middle	$O(n)$	Shift elements
Delete from middle	$O(n)$	Shift elements

Array Types

Type	Size	Language Example
Static	Fixed at compile	C: <code>int arr[100]</code>
Dynamic	Grows at runtime	Java: <code>ArrayList</code> , Python: <code>list</code>
2D	Grid/matrix	C: <code>int mat[3][4]</code>

Traps

Trap 1: Index Out of Bounds - C: No bounds checking, may access garbage - Java/Python: Throws `ArrayIndexOutOfBoundsException`

Trap 2: Off-By-One Error - Arrays are 0-indexed: indices 0 to $n-1$ - Accessing `arr[n]` is always out of bounds

Trap 3: Array Name is NOT a Pointer (C) - `sizeof(arr)` gives total array size, not pointer size - Array decays to pointer in most contexts

1.2 Stacks

Definition

A **stack** is a LIFO (Last-In, First-Out) data structure where the last element added is the first removed.

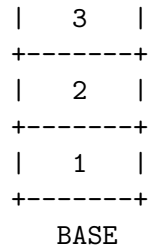
Stack Operations

Operation	Description	Complexity
push(item)	Add to top	$O(1)$
pop()	Remove from top	$O(1)$
peek()	View top without removing	$O(1)$
isEmpty()	Check if empty	$O(1)$

LIFO Visualization

```

+-----+
|  5  | <- TOP (last in, first out)
+-----+
|  4  |
+-----+
```



Stack Applications

Application	Description
Function Call Stack	Manage recursion and function calls
Undo/Redo	Store operation history
Expression Evaluation	Evaluate postfix, infix to postfix
Parentheses Matching	Check balanced brackets
Backtracking	DFS traversal

Traps

Trap: Stack Overflow - Unlimited recursion causes stack overflow - Every recursive function needs a base case

Trap: Stack Underflow - Popping from empty stack throws exception - Always check `isEmpty()` before pop

1.3 Queues

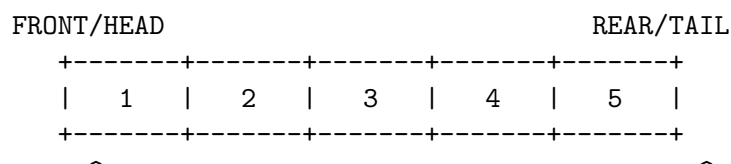
Definition

A **queue** is a FIFO (First-In, First-Out) data structure where the first element added is the first removed.

Queue Operations

Operation	Description	Complexity
enqueue(item)	Add to rear	O(1)
dequeue()	Remove from front	O(1)
peek()	View front without removing	O(1)

FIFO Visualization



first in
first out

last in
last out

Queue Types

Type	Description
Simple Queue (FIFO)	Standard first-in-first-out
Circular Queue	Wraps around end to beginning
Priority Queue	Highest priority dequeued first
Deque (Double-ended)	Add/remove from both ends

Applications

Application	Description
BFS (Breadth-First Search)	Graph traversal
Task Scheduling	Print queue, CPU scheduling
Buffer	Network packets, I/O operations

Traps

Trap: Array Queue Wastes Space - Simple array queue fills up even after dequeue - Solution: Use circular queue

1.4 Linked Lists

Definition

A **linked list** is a linear data structure where elements (nodes) are connected by pointers. Each node contains data and a pointer to the next node.

Types of Linked Lists

Type	Pointer Direction	Notes
Singly Linked	Forward only	Simple, memory efficient
Doubly Linked	Forward and backward	Easier deletion
Circular	Last points to first	Ring structure

Linked List vs Array

Aspect	Linked List	Array
Memory	Scattered	Contiguous
Access	$O(n)$	$O(1)$
Insert at head	$O(1)$	$O(n)$

Aspect	Linked List	Array
Insert at tail	O(1) with tail pointer	O(1) amortized
Memory overhead	Higher (pointers)	Lower
Cache locality	Poor	Excellent

Floyd's Cycle Detection

Two pointers (slow and fast): - Slow moves 1 step, fast moves 2 steps - If they meet, cycle exists - Time: O(n), Space: O(1)

Traps

Trap: Losing Head Pointer - Must update head when inserting at head

Trap: Null Pointer in Loop - Always check if list is empty before traversing

Trap: Memory Leak (C/C++) - Must free nodes when deleting

1.5 Tower of Hanoi

Problem

Move n disks from Source to Destination peg using Auxiliary peg, with rules: 1. Only one disk at a time 2. Larger disk cannot be on smaller 3. Only top disk can be moved

Solution Analysis

Aspect	Value
Number of moves	$2^n - 1$
Time complexity	$O(2^n)$ exponential
Space complexity	$O(n)$ recursion depth

Recursive Solution

```
tower(n-1, source, auxiliary, destination) // Move n-1 to auxiliary
move largest from source to destination
tower(n-1, auxiliary, destination, source) // Move n-1 to destination
```

Traps

Trap: Exponential Time - For n=64, moves = $2^{64} - 1 \sim 1.8 \times 10^{19}$ - At 1 move/second, takes 584 billion years!

Chapter 2: Trees and Hierarchical Structures

2.1 Tree Fundamentals

Definition

A **tree** is a hierarchical data structure with exactly one path between any two nodes.

Key Terminology

Term	Definition
Root	Topmost node (no parent)
Leaf	Node with no children
Parent	Node with children
Child	Node with a parent
Sibling	Nodes with same parent
Depth	Edges from root to node
Height	Longest path from node to leaf
Subtree	Node and its descendants

Tree vs Linear Structures

Aspect	Linear	Tree
Structure	One path	Hierarchical
Access	Direct (array) or sequential	$O(n)$ worst case
Search	$O(n)$ or $O(\log n)$	$O(\log n)$ average
Order	No inherent	Sorted (BST)

2.2 Binary Trees

Definition

A **binary tree** is a tree where each node has at most two children: left and right.

Binary Tree Properties

Max nodes at level $i = 2^i$

Max nodes in tree of height $h = 2^{(h+1)} - 1$

Min height with n nodes = $\text{floor}(\log_2(n))$

Types of Binary Trees

Type	Definition
Full/Strict	Every node has 0 or 2 children
Perfect	All internal nodes have 2 children, all leaves same level

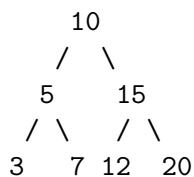
Type	Definition
Complete	All levels filled except possibly last, filled left to right
Balanced	Left and right subtrees height differ by at most 1
Degenerate	Each parent has only one child (like linked list)

2.3 Tree Traversals

The Four Main Traversals

Traversal	Order	Use Case
Inorder (LNR)	Left, Node, Right	BST sorted output
Preorder (NLR)	Node, Left, Right	Copy tree, prefix
Postorder (LRN)	Left, Right, Node	Delete tree, postfix
Level Order	Level by level	BFS, shortest path

Traversal Results Example



Inorder: 3, 5, 7, 10, 12, 15, 20 (sorted for BST)
 Preorder: 10, 5, 3, 7, 15, 12, 20
 Postorder: 3, 7, 5, 12, 20, 15, 10
 Level: 10, 5, 15, 3, 7, 12, 20

Traps

Trap: Null Pointer in Traversal - Every recursive traversal **MUST** check for null first

Trap: Using Wrong Traversal for BST - Only inorder gives sorted output

2.4 Binary Search Trees (BST)

Definition

A **BST** is a binary tree where: - Left subtree contains nodes less than current node - Right subtree contains nodes greater than current node - Both subtrees must be BSTs

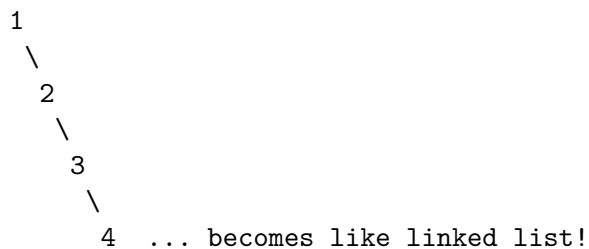
BST Operations

Operation	Average	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Find Min	$O(\log n)$	$O(n)$
Find Max	$O(\log n)$	$O(n)$

Delete Operation Cases

Case	Description	Action
Leaf	No children	Remove directly
One Child	One subtree	Replace with child
Two Children	Both subtrees exist	Replace with inorder successor

Worst Case (Degenerate BST)



Height = n , Search = $O(n)$ - NOT balanced!

Traps

Trap: BST vs Binary Tree - Binary tree is NOT a BST unless $\text{left} < \text{node} < \text{right}$ for ALL nodes

Trap: No Duplicates - BST typically doesn't allow duplicates (check spec!)

2.5 AVL Trees

Definition

An **AVL tree** is a self-balancing BST where heights of left and right subtrees differ by at most 1.

Balance Factor

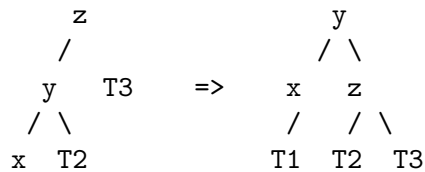
`balance_factor = height(left) - height(right)`
 Must be: -1, 0, or 1

Rotations Needed

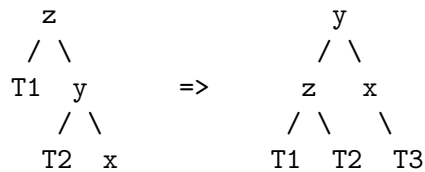
Imbalance	Rotation	Pattern
LL	Right rotation	Left child of left child
RR	Left rotation	Right child of right child
LR	Left then Right	Left child of right child
RL	Right then Left	Right child of left child

Single Rotation Examples

LL Case (Right Rotation):



RR Case (Left Rotation):



AVL vs BST

Aspect	BST	AVL
Balance	None	Self-maintained
Search	$O(h)$	$O(\log n)$ guaranteed
Insert/Delete	$O(h)$	$O(\log n)$ + rotations
Height	Can be $O(n)$	Always $O(\log n)$

2.6 Heaps

Definition

A **heap** is a complete binary tree that satisfies the heap property: - **Max Heap**: Parent $\geq$ children (largest at root) - **Min Heap**: Parent $\leq$ children (smallest at root)

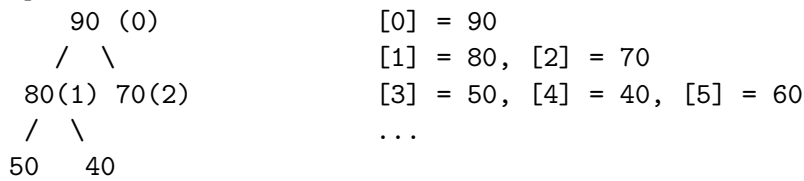
Heap as Array (0-indexed)

Parent at i : $(i - 1) / 2$

Left child: $2 * i + 1$

Right child: $2 * i + 2$

Example:



Heap Operations

Operation	Complexity	Description
Insert	$O(\log n)$	Add to end, bubble up
Extract Min/Max	$O(\log n)$	Remove root, bubble down
Get Min/Max	$O(1)$	Root element
Heapify	$O(n)$	Build heap from array

Heap vs BST

Property	Heap	BST
Order	Parent $>$ Children	Left $<$ Root $<$ Right
Find Min/Max	$O(1)$	$O(\log n)$
Sorted Traversal	Not possible efficiently	Inorder = sorted
Structure	Always complete	Any shape

Traps

Trap: Heap is NOT a BST! - Heap only guarantees parent $>$ children - Left/right relationship is not guaranteed

Trap: Array Index Formulas - For 0-indexed: parent = $(i-1)/2$, left = $2i+1$ - For 1-indexed: parent = $i/2$, left = $2i$

2.7 B+ Trees

Definition

A **B+ Tree** is a self-balancing tree where: - All data (values) stored only at leaf nodes - Internal nodes only contain keys for routing - Leaf nodes are linked sequentially

B+ Tree vs B-Tree vs BST

Aspect	BST	B-Tree	B+ Tree
Height	Can be $O(n)$	Logarithmic, flat	Logarithmic, very flat
Storage	One key per node	Keys + values in all nodes	Only values in leaves
Range queries	$O(n)$	$O(n)$	$O(\log n + k)$

Aspect	BST	B-Tree	B+ Tree
Leaf linking	No	No	Yes

B+ Tree Properties

Property	Value
Order (m)	Maximum children per node
Max keys in leaf	$m - 1$
All leaves same depth	Yes
Leaves linked	Yes (for range queries)

Why B+ Trees Used in Databases

Advantages:

1. FLAT STRUCTURE: Fewer disk reads
2. HIGH FANOUT: Hundreds of keys per node
3. LEAF LINKING: Efficient range queries

Traps

Trap: B-Tree vs B+ Tree - B-Tree: Values stored in ALL nodes - B+ Tree: Values stored ONLY in leaf nodes

Trap: Order m means m-1 keys - If order $m = 5$, max keys per leaf = 4

Chapter 3: Hashing Techniques

3.1 Hash Tables

Definition

A **hash table** stores key-value pairs and provides $O(1)$ average-case lookup, insertion, and deletion using a **hash function**.

Why Hashing?

Data Structure	Search	Insert	Delete
Array (unsorted)	$O(n)$	$O(1)^*$	$O(n)$
BST	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg

*At end of structure

Hash Table vs BST

Aspect	Hash Table	BST
Search complexity	$O(1)$ average	$O(\log n)$
Order	No ordering	Sorted
Worst case	$O(n)$	$O(\log n)$
Implementation	Simpler	More complex

3.2 Hash Functions

Definition

A **hash function** converts a key into an index in the hash table.

Good Hash Function Properties

1. **Deterministic:** Same key always produces same hash
2. **Uniform distribution:** Hashes spread uniformly
3. **Fast:** $O(1)$ computation
4. **Minimize collisions:** Different keys should map to different indices

Common Hash Functions

Method	Formula	Notes
Division	$\text{key} \% \text{tableSize}$	Simple, use prime table size
Multiplication	$\text{floor}(m * (\text{key} * A \bmod 1))$	A is constant (golden ratio)
String	Sum of $(\text{char} * 31^{\text{position}})$	For string keys

Good Table Size

- Should be a **prime number**
- Not a power of 2
- Large enough to keep load factor low

3.3 Collisions

Definition

A **collision** occurs when two different keys hash to the same index.

Collision Resolution Strategies

Strategy	How it Works
Chaining	Linked list at each slot

Strategy	How it Works
Open Addressing	Find next empty slot

3.4 Chaining (Separate Chaining)

How it Works

Each slot contains a linked list of items that hash to that index.

Time Complexity

Operation	Average	Worst
Search	$O(1 + \text{load factor})$	$O(n)$
Insert	$O(1)$	$O(n)$
Delete	$O(1 + \text{load factor})$	$O(n)$

Load Factor and Rehashing

`load_factor = n / tableSize`

When `load_factor > 0.75`: REHASH!

Rehashing: Create larger table, reinsert all elements

3.5 Open Addressing

Definition

All items stored IN the table itself. When collision occurs, find another open slot.

Types of Probing

Type	Strategy
Linear Probing	Check consecutive slots: $h, h+1, h+2 \dots$
Quadratic Probing	Check with quadratic jumps: $h, h+1^2, h+2^2 \dots$
Double Hashing	Use second hash function: $h_1 + i \cdot h_2$

Linear Probing - Primary Clustering

Problem: Long contiguous clusters form

New items that hash near the cluster extend it

This makes big clusters grow bigger (positive feedback)

Quadratic Probing - Secondary Clustering

Problem: Items with same initial hash probe same sequence

Two items hashing to index 3 will both try 4, 7, 12...

Even though they're different keys

Double Hashing

Probe sequence: $h(\text{key}, i) = (h1(\text{key}) + i * h2(\text{key})) \% \text{tableSize}$

Best clustering avoidance because second hash affects probe sequence

Comparison of Methods

Method	Pros	Cons
Chaining	Handles unlimited items	Extra memory for lists
Linear Probing	Simple, cache-friendly	Primary clustering
Quadratic Probing	Less clustering	Secondary clustering
Double Hashing	Best distribution	More computation

3.6 HashMap in Java

Key Methods

Method	Complexity
put(key, value)	O(1) average
get(key)	O(1) average
remove(key)	O(1) average
containsKey(key)	O(1) average

HashMap vs Hashtable

Aspect	HashMap	Hashtable
Thread safety	Not synchronized	Synchronized
Null keys	Allows one	No
Null values	Allows	No

Chapter 4: Graphs

4.1 Graph Fundamentals

Definition

A **graph** consists of vertices (nodes) connected by edges.

Graph Terminology

Term	Definition
Vertex (Node)	A point in the graph
Edge	Connection between vertices
Adjacent vertices	Connected by an edge
Degree	Number of edges at a vertex
In-degree	Edges coming INTO vertex (digraph)
Out-degree	Edges going OUT of vertex (digraph)
Path	Sequence of vertices
Cycle	Path returning to starting vertex

Types of Graphs

Type	Description
Undirected	Edges have no direction
Directed (Digraph)	Edges have direction
Weighted	Edges have weights/costs
Cyclic	Contains cycles
Acyclic	No cycles
Complete	All pairs connected

DAG (Directed Acyclic Graph)

A directed graph with no cycles. Important for: - Task scheduling - Dependency resolution - Topological ordering

4.2 Graph Representations

Adjacency Matrix

2D array where `matrix[i][j]` = weight if edge exists.

Aspect	Value
Space	$O(V^2)$
Add edge	$O(1)$
Remove edge	$O(1)$
Check edge	$O(1)$
Find neighbors	$O(V)$
Best for	Dense graphs

Adjacency List

Array of lists where each list contains neighbors.

Aspect	Value
Space	$O(V + E)$
Add edge	$O(1)$
Remove edge	$O(V)$
Check edge	$O(V)$
Find neighbors	$O(\text{degree})$
Best for	Sparse graphs

When to Use Which

Use Matrix	Use List
Dense graph (E close to V^2)	Sparse graph
Need $O(1)$ edge lookup	Iterate neighbors frequently
Quick edge existence check	Memory is limited

4.3 Breadth-First Search (BFS)

Definition

BFS explores vertices in “layers” - visits all neighbors before going deeper. Uses a **queue**.

BFS Characteristics

Aspect	Value
Data structure	Queue
Exploration	Layer by layer
Shortest path	Yes (unweighted)
Time complexity	$O(V + E)$
Space complexity	$O(V)$

BFS Order

Graph:	BFS from A:
A	Level 0: A
/ \	Level 1: B, C, D
B C D	Level 2: E, F

Order: A, B, C, D, E, F

BFS Applications

Application	Use Case
Shortest path	Unweighted graph shortest path
Level order traversal	Tree level-by-level
Connected components	Find all reachable nodes
Bipartite checking	Can graph be 2-colored?

4.4 Depth-First Search (DFS)

Definition

DFS explores as far as possible along each branch before backtracking. Uses a **stack** (or recursion).

DFS Characteristics

Aspect	Value
Data structure	Stack (or recursion)
Exploration	Deep along each branch
Shortest path	No
Time complexity	$O(V + E)$
Space complexity	$O(V)$

DFS Applications

Application	Use Case
Topological sort	Ordering DAG vertices
Cycle detection	For directed graphs
Path finding	Find any path
Connected components	Find SCCs

BFS vs DFS

Aspect	BFS	DFS
Data structure	Queue	Stack
Shortest path	Yes	No
Memory	$O(V) + O(V)$ queue	$O(V)$ stack
When to use	Shortest path	Path finding, topological sort

4.5 Minimum Spanning Tree (MST)

Definition

A **spanning tree** connects all vertices with $V-1$ edges and no cycles. A **Minimum Spanning Tree** has minimum total weight.

MST Properties

Property	Value
Unique	If all edge weights are distinct
Number of edges	$V - 1$
No cycles	Yes
Minimum total	Yes

Prim's Algorithm

Algorithm:

1. Start with any vertex
2. At each step, add cheapest edge from tree to non-tree vertex
3. Repeat until all vertices in tree

Time: $O(V^2)$ or $O(E \log V)$ with priority queue

Kruskal's Algorithm

Algorithm:

1. Sort all edges by weight
2. Add edges from smallest, skipping those that create cycle
3. Stop when $V-1$ edges added

Time: $O(E \log E)$

Uses Union-Find data structure

Prim vs Kruskal

Aspect	Prim	Kruskal
Approach	Grow from one vertex	Grow from edges
Data structure	Priority queue	Sort + Union-Find
Time (dense)	$O(V^2)$ better	$O(E \log E)$ worse
Time (sparse)	$O(E \log V)$	$O(E \log E)$
Best for	Dense graphs	Sparse graphs

4.6 Shortest Path Algorithms

Dijkstra's Algorithm

Finds shortest paths from source to all vertices. Works only with non-negative weights.

Algorithm:

1. Initialize distances: source = 0, others = INF
2. Mark all vertices unvisited
3. While unvisited vertices remain:

- a. Pick vertex with minimum distance
- b. Mark as visited
- c. Update distances to neighbors

Aspect	Value
Time (simple)	$O(V^2)$
Time (with PQ)	$O((V+E) \log V)$
Negative weights	DOES NOT WORK

Dijkstra vs Bellman-Ford

Aspect	Dijkstra	Bellman-Ford
Negative weights	No	Yes
Time complexity	$O((V+E) \log V)$	$O(V * E)$
Negative cycle detection	No	Yes

Bellman-Ford Algorithm

Why it works:

- Any simple path has at most $V-1$ edges
- After $V-1$ iterations, all shortest paths are found

For negative weight detection:

- If any distance can still be reduced after $V-1$ iterations, a negative cycle exists

When to Use

Scenario	Algorithm
All weights positive	Dijkstra (faster)
Some negative weights	Bellman-Ford
Need negative cycle detection	Bellman-Ford

4.7 Topological Sort

Definition

Topological sort orders vertices in a DAG so that for every edge (u, v) , u comes before v .

Valid Topological Orders

For course prerequisites:

CS101 $\rightarrow$ CS201 $\rightarrow$ CS301

Valid orders:

CS101, CS201, CS301

Any order respecting prerequisites

Two Methods

Method	Approach
Kahn's Algorithm	BFS with in-degree tracking
DFS + Stack	Push after exploring all dependencies

Topological Sort Applications

Application	Example
Course scheduling	Prerequisites before courses
Build systems	Compile order for files
Task scheduling	Order dependent tasks

4.8 Cycle Detection

Undirected Graph Cycle Detection

Using DFS: If we find a visited neighbor that is not our parent, cycle exists.

Directed Graph Cycle Detection

Using DFS with 3 states: - UNVISITED: Not yet visited - IN_PROGRESS: Currently in recursion stack - VISITED: Fully processed

If we find IN_PROGRESS neighbor, cycle exists.

Practice Questions and Solutions

MCQ 1: Array Access

What is the time complexity of accessing element at index i in an array? - A) $O(n)$ - B) $O(1)$ - C) $O(\log n)$ - D) $O(n^2)$

Answer: B) $O(1)$ - Direct address calculation

MCQ 2: Stack

What data structure uses LIFO? - A) Queue - B) Stack - C) Linked List - D) Array

Answer: B) Stack

MCQ 3: BST

What is the worst case search time in a BST with n nodes? - A) $O(1)$ - B) $O(\log n)$ - C) $O(n)$ - D) $O(n \log n)$

Answer: C) $O(n)$ - Degenerate tree (like linked list)

MCQ 4: AVL

What is the maximum height difference between left and right subtrees in an AVL tree? - A) 0 - B) 1 - C) 2 - D) Any value

Answer: B) 1 - AVL balance factor is -1, 0, or 1

MCQ 5: Heap

In a max heap, which node has the largest value? - A) Any leaf - B) Root - C) Leftmost node - D) Rightmost node

Answer: B) Root - Max heap: root is always maximum

MCQ 6: Hash Collision

Two different keys produce the same hash value. This is called: - A) Overloading - B) Collision - C) Overflow - D) Redundancy

Answer: B) Collision

MCQ 7: BFS

What data structure does BFS use? - A) Stack - B) Queue - C) Priority Queue - D) Deque

Answer: B) Queue

MCQ 8: Dijkstra

Dijkstra's algorithm works for graphs with: - A) Any weights - B) Only positive weights - C) Only negative weights - D) No weights

Answer: B) Only positive weights - Negative weights cause wrong results

MCQ 9: MST

A minimum spanning tree with V vertices has how many edges? - A) V - B) $V - 1$ - C) $2V$ - D) V^2

Answer: B) $V - 1$ - MST connects all vertices with no cycles

MCQ 10: Topological Sort

Topological sort can only be performed on: - A) Any graph - B) DAG (Directed Acyclic Graph) - C) Undirected graph - D) Complete graph

Answer: B) DAG - Topological sort requires no cycles

Short Answer 1

Explain why arrays have $O(1)$ random access while linked lists have $O(n)$.

Answer: Arrays store elements in contiguous memory, so the address of element i is calculated directly as $\text{base_address} + i * \text{element_size}$. This is a single arithmetic operation. Linked lists scatter nodes in memory, so to reach node i you must start at the head and follow i pointers sequentially, requiring i operations.

Short Answer 2

Why do we prefer BST over array for sorted data insertion?

Answer: Array insertion in sorted order is $O(n)$ because elements must be shifted to make room. BST insertion is $O(\log n)$ on average because we only traverse one path from root to insertion point. For large datasets, BST is much faster for dynamic sorted data.

Short Answer 3

Why can't Dijkstra's algorithm handle negative edge weights?

Answer: Dijkstra's greedy approach assumes that once a vertex is visited with minimum distance, that distance is final. With negative weights, a shorter path could later be discovered through an unvisited vertex. The algorithm would incorrectly mark the vertex as finalized with the wrong distance.

Short Answer 4

Explain primary clustering in linear probing.

Answer: Primary clustering occurs when items that hash to nearby indices form long contiguous sequences (clusters). New items that hash anywhere near an existing cluster will probe through it, causing the cluster to grow even larger. This creates a positive feedback loop where big clusters attract more items and grow bigger, degrading performance.

Short Answer 5

Why are B+ trees preferred over BST for database indexing?

Answer: B+ trees are preferred because: (1) They are much flatter - with high fanout, millions of records can be accessed with just 3-4 disk reads vs 20+ for BST; (2) All data is stored at leaves which can be sequentially accessed; (3) The flat structure means fewer disk I/O operations which are expensive; (4) Tree height is guaranteed to be logarithmic and very shallow.

Short Answer 6

Explain the difference between BFS and DFS.

Answer: BFS (Breadth-First Search) uses a queue and explores vertices level by level - all neighbors before going deeper. It's guaranteed to find shortest path in unweighted graphs. DFS (Depth-First Search) uses a stack (or recursion) and explores as deep as possible along each branch before backtracking. DFS is better for path finding, topological sorting, and uses less memory for wide graphs.

Cheat-Sheet: Time Complexities

Data Structures

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)^*$	$O(n)$
Stack	Top only	$O(n)$	$O(1)$	$O(1)$
Queue	Front only	$O(n)$	$O(1)$	$O(1)$
Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
BST (avg)	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
BST (worst)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AVL	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	$O(1)$	$O(n)$	$O(\log n)$	$O(\log n)$
Hash Table	-	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg

*With head/tail pointer

Graph Algorithms

Algorithm	Time	Space	Use Case
BFS	$O(V+E)$	$O(V)$	Shortest path (unweighted)
DFS	$O(V+E)$	$O(V)$	Path finding, topo sort
Dijkstra	$O((V+E) \log V)$	$O(V)$	Shortest path (positive)
Bellman-Ford	$O(VE)$	$O(V)$	Shortest path (any weight)
Prim	$O((V+E) \log V)$	$O(V)$	MST
Kruskal	$O(E \log E)$	$O(V)$	MST

Review Checklist

Arrays and Lists

- ☐ Can you explain $O(1)$ access calculation?
- ☐ Do you know static vs dynamic arrays?
- ☐ Can you compare array vs linked list?

Stacks and Queues

- ☐ Do you understand LIFO vs FIFO?
- ☐ Can you name stack/queue applications?
- ☐ Do you know when to use which?

Trees

- ☐ Can you distinguish tree types (full, perfect, complete)?
- ☐ Do you know all 4 traversals and when to use each?
- ☐ Can you explain BST search/insert/delete?
- ☐ Do you understand AVL rotations (LL, RR, LR, RL)?
- ☐ Can you compare heap vs BST?
- ☐ Do you understand B+ tree structure?

Hashing

- ☐ Can you explain hash function properties?
- ☐ Do you know collision resolution methods?
- ☐ Can you explain clustering problems?
- ☐ Do you understand load factor and rehashing?

Graphs

- ☐ Can you represent graphs (matrix vs list)?
- ☐ Do you know BFS vs DFS characteristics?
- ☐ Can you explain MST algorithms (Prim, Kruskal)?
- ☐ Do you know Dijkstra vs Bellman-Ford?
- ☐ Can you perform topological sort?
- ☐ Do you understand cycle detection?